S1.01 — Implémentation d'un besoin client : Méthodologie, Modularité et Contrats

Thème principal : Spécification, conception modulaire et robustesse logicielle

Synthèse exécutive : Ce cours pose les fondations méthodologiques indispensables du premier projet intégrateur (SAÉ S1.01) du BUT Informatique. Il explique comment transformer un besoin client informel en spécifications fonctionnelles rigoureuses, structurer un programme en appliquant la décomposition descendante (Top-Down), séparer strictement la logique métier des interfaces graphiques ou textuelles, et garantir la robustesse du code via la programmation par contrat, les assertions et les tests unitaires.

De l'Expression du Besoin à la Spécification Fonctionnelle

En informatique professionnelle, un projet échoue rarement à cause d'une incapacité à coder, mais presque toujours à cause d'un décalage entre ce que le client a exprimé, ce que le développeur a compris, et ce qui a été effectivement testé. L'objectif initial consiste donc à transformer un énoncé informel en un ensemble de règles explicites. Toute ambiguïté lexicale doit être bannie au profit de seuils quantifiables et de comportements déterministes.

- Expression de besoin : formulation subjective et parfois contradictoire fournie par l'utilisateur final ou le client
- Spécification fonctionnelle : traduction technique univoque décrivant le comportement attendu du système sans présager de son implémentation
- Critères d'acceptation : conditions observables permettant d'arbitrer sans équivoque si une fonctionnalité est terminée et valide via le triptyque 'Étant donné / Quand / Alors'

Cas concret enseignant : *L'enseignant illustre ce problème avec l'analogie de la 'balançoire d'entreprise' : le client demande une balançoire suspendue à une branche d'arbre, mais chaque corps de métier l'interprète différemment jusqu'à couper le tronc de l'arbre en deux pour faire passer la corde, alors que le client voulait simplement un pneu attaché avec un câble.*

Décomposition Modulaire et Conception Descendante

Pour concevoir un programme robuste dans le cadre de la SAÉ S1.01, l'étudiant doit proscrire la rédaction d'un bloc de code monolithique dans une seule fonction 'main'. Un problème global P est découpé en sous-tâches P1, P2, ..., Pn. On sépare systématiquement la logique métier (calculs, algorithmes purs) de la couche d'interface utilisateur (affichages écran et saisies clavier).

- Conception descendante (Top-Down) : démarche consistant à scinder un problème complexe en sous-problèmes élémentaires indépendants
- Modularité : découpage du code en modules autonomes présentant une forte cohésion interne et un faible couplage avec les autres modules
- Principe de responsabilité unique (Single Responsibility Principle) : chaque fonction ne doit réaliser qu'une seule tâche précise

Cas concret enseignant : *Métaphore de la chaîne de montage automobile : le technicien qui assemble l'alternateur n'a pas besoin de savoir quelle est la couleur des sièges en cuir. Si une fonction de calcul fiscal contient des instructions d'affichage à l'écran, elle devient inutilisable sur un serveur web ou dans une application mobile.*

Programmation par Contrat et Stratégie de Test

Une fonction mathématique ou informatique n'est fiable que si son domaine de validité est formellement délimité. La programmation par contrat stipule que si l'appelant respecte les préconditions, la fonction s'engage à satisfaire les postconditions. Les assertions permettent de stopper immédiatement l'exécution lors de la phase de débogage si

une précondition est violée en interne.

- Précondition : hypothèse indispensable sur les paramètres d'entrée pour que la fonction garantisse un résultat correct
- Postcondition : état du système et valeur de retour garantis par la fonction une fois son exécution terminée
- Test unitaire : fonction automatisée vérifiant qu'une unité de code (fonction isolée) réagit conformément aux spécifications

Cas concret enseignant : *Le professeur insiste sur l'accident historique de la fusée Ariane 5 (vol 501 en 1996) : l'explosion a été provoquée par une conversion d'un nombre flottant 64 bits en entier 16 bits sans vérification des préconditions sur la plage de valeurs admissibles.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi ne peut-on pas mettre de 'printf' ou d'affichages dans nos fonctions de calcul d'algorithmes ?
Réponse du professeur : Parce que cela crée un couplage fort avec la console. Une fonction de calcul doit être pure et réutilisable : si on souhaite l'intégrer dans une application graphique, web ou mobile, la présence de 'printf' bloquera l'exécution ou affichera du texte au mauvais endroit.

Question étudiant : Quelle est la différence exacte entre une précondition et une condition de validation de saisie utilisateur ?
Réponse du professeur : La validation de saisie intercepte une erreur humaine au clavier et redemande la valeur à l'utilisateur. La précondition est un contrat interne entre développeurs : si elle est violée, c'est qu'il y a un bug de programmation en amont, d'où l'utilisation d'une assertion pour crasher immédiatement le programme en mode debug.

■ Exercice & Démonstration d'Application

Énoncé : Spécifier et implémenter une fonction modulaire 'calculer_moyenne_ponderee' qui prend en entrée une liste de notes (flottants sur 20) et une liste de coefficients entiers correspondants, puis renvoie la moyenne pondérée.

Démarche de résolution :
1. Définition des préconditions : taille > 0, notes entre 0.0 et 20.0, coefficients > 0, somme des coefficients non nulle. 2. Initialisation des accumulateurs somme_points et somme_coeffs à 0. 3. Boucle de parcours de 0 à taille-1 pour cumuler les produits note*coefficient et la somme des coefficients. 4. Division sécurisée de la somme des points par la somme des coefficients.

```c
/* ------------------------------------------------------------------------ */
/* Fichier : moyenne_ponderee.c                                             */
/* Auteur : Matéi de Saint Leger (BUT 1 Info Anglet)                        */
/* Rôle : Calcul modulaire d'une moyenne pondérée sécurisée par contrat     */
/* ------------------------------------------------------------------------ */

#include <stdio.h>
#include <assert.h>

// CONSTANTES
#HAVE_LIMITS

// VARIABLES
// (clavier) >> saisir >> notes, coefficients, taille

float calculer_moyenne_ponderee(const float tab_notes[], const int tab_coeffs[], int taille)
{
    // PRECONDITIONS
    assert(taille > 0);

    // VARIABLES
    int i;
    float somme_points;
    int somme_coeffs;
    float moyenne;

    // TRAITEMENTS
    somme_points = 0.0f;
    somme_coeffs = 0;

    for (i = 0; i < taille; i++)
    {
        assert(tab_notes[i] >= 0.0f && tab_notes[i] <= 20.0f);
        assert(tab_coeffs[i] > 0);

        somme_points += tab_notes[i] * tab_coeffs[i];
        somme_coeffs += tab_coeffs[i];
    }

    assert(somme_coeffs > 0);
    moyenne = somme_points / somme_coeffs;

    // POSTCONDITION implicite : renvoi d'un flottant valide
    return moyenne;
}
```

Résultat : La fonction retourne un flottant représentant la moyenne pondérée exacte sans aucun couplage avec les flux d'entrées/sorties console.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque majeur d'un couplage fort entre le traitement algorithmique et l'affichage console ?

R : L'impossibilité de réutiliser la fonction dans un autre contexte (interface graphique, web, API) et la difficulté à automatiser les tests unitaires.

Q : Qu'appelle-t-on un cas limite (edge case) en conception algorithmique ?

R : Une situation aux frontières du domaine de validité des données (ex: tableau vide, valeur minimale ou maximale admise, division par zéro).

Q : À quoi sert une assertion (assert) dans un sous-programme ?

R : À vérifier formellement qu'une précondition ou une hypothèse interne est vraie, et à interrompre immédiatement le programme en phase de développement si elle est fausse.